Checkpoint 5 · Avaliação Teórica

FIAP Bank – Plataforma de Concessão de Crédito Domain Driven Design - Java · Turma 2ESPG · Prof. Eduardo dos Santos Ramos

Integrante	RM
Gabriel Viana de Souza	RM 564382
Rafael Falaguasta Ferraz	RM 561714

Diagnóstico das falhas do estado inicial (Fase 2)

#	Onde	O que estava errado	Por que é um problema	Correção aplicada
1	`pom.xml` da Plataforma e da Mesa	Spring Cloud `2024.0.0` com Spring Boot `4.1.1`	O release train 2024.0 só suporta Boot 3.4. Com Boot 4.1 o OpenFeign é incompatível e a aplicação não sobe	Spring Cloud `2025.1.3` (Oakwood), compatível com Boot 4.0/4.1
2	`CoreBancarioClientService`	`catch (Exception)` devolvia **sempre** a conta fictícia `12345-6` com saldo R$ 12.500,00, para qualquer CPF	Mascara o Core fora do ar ou o cliente sem conta e alimenta a análise com dados falsos	Adapter anticorrupção: 404 vira "sem conta", falha de rede vira `IntegracaoIndisponivelException`, e o motor encaminha a proposta para a Mesa
3	`PlataformaCreditoService.solicitarCredito`	Consultava o Core, mas a variável `contaCore` nunca era usada	A regra de negócio "validar saldos e movimentações no Core" não existia de fato	Regra 5 do `MotorRegrasCredito`: sem conta ativa ou Core indisponível → `EM_ANALISE`
4	Fluxo de análise	O CRM corporativo não era consultado	O business case exige o histórico do cliente no CRM	Porta `CrmGateway` + `CrmGatewayAdapter`
5	`fiap-bank-core-bancario`	Projeto inexistente (só README)	Toda chamada Feign ao Core falhava e caía no fallback falso	Core Bancário construído do zero (Fase 1)
6	Service + controller	"Não encontrado" lançava `IllegalArgumentException`, sem tratamento global	O cliente recebia **HTTP 500** para um ID inexistente	`RecursoNaoEncontradoException` → **404** via `@RestControllerAdvice` (Problem Details)
7	DTOs de entrada	Nenhuma validação	`quantidadeParcelas = 0` gerava `ArithmeticException` (divisão por zero) e campos nulos geravam `NullPointerException`, ambos HTTP 500	Bean Validation nos records + invariantes no agregado → **400**
8	README × DTOs	O README documentava `{"id": 1}` e `parecerOperador` na reprovação, mas os DTOs esperavam `solicitacaoId` e `motivoReprovacao`	A requisição documentada chegava com ID nulo e quebrava	O adapter legado aceita os dois nomes (`@JsonAlias`)
9	Aprovar/Reprovar	Nenhuma verificação de status	Era possível "aprovar" uma proposta já reprovada pelo SCR	Invariante no agregado: só propostas pendentes recebem decisão (**409** caso contrário)
10	Spring Boot 4	`spring-boot-starter-web` depreciado e console H2 sem o módulo `spring-boot-h2console`	O console H2 prometido no README não subia	`spring-boot-starter-webmvc` + `spring-boot-h2console`
11	Feign client	Retorno `Map<String, Object>`	Contrato não tipado: erros só aparecem em runtime	Record tipado `ContaCoreResponse`
12	Logs	`System.out.println` / `System.err.println`	Sem níveis, sem integração com observabilidade	SLF4J (`@Slf4j`)

> Observação: o seed usa `tipoCredito = "EMPRESTIMO_PESSOAL"`, enquanto o app Mobile envia `"EMPRÉSTIMO PESSOAL"`. Mantivemos `tipoCredito` como texto livre validado (e não como enum) para não quebrar o contrato do Mobile.

1. Quais componentes violam o isolamento do domínio e misturam código de infraestrutura relacional com regras de negócio?

R: O componente que viola é a **Plataforma de Crédito** (`fiap-bank-plataforma-credito`), em especial:

- **`model/SolicitacaoCredito`**: é, ao mesmo tempo, o modelo de negócio e a entidade JPA (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@Enumerated`). O domínio fica acoplado ao Hibernate e ao esquema do banco: mudar uma coluna mexe no modelo de negócio, e não dá para testar a regra sem JPA. Além disso, o modelo é **anêmico**: só tem getters e setters públicos, e qualquer classe pode colocar a proposta em qualquer status (por exemplo, aprovar uma proposta já reprovada), sem proteção de

invariantes.

- **`service/PlataformaCreditoService`**: concentra o motor de regras de crédito (SCR, score, comprometimento de 30% da renda) junto com orquestração, persistência via `JpaRepository` (Spring Data) e chamadas de integração. A regra de negócio fica presa num `@Service` do Spring, misturada com infraestrutura, quando deveria estar em entidades e Domain Services puros.
- **`dto/SolicitacaoCreditoResponseDTO.fromEntity(...)`**: o contrato da API depende diretamente da entidade JPA, acoplando apresentação e persistência.
- **`repository/SolicitacaoCreditoRepository extends JpaRepository`**: não existe uma porta de repositório do domínio. O "repositório" é a própria interface do framework, então o domínio não fica desacoplado da tecnologia de persistência.
- **`service/ScrService`, `BureauCreditoService`, `CoreBancarioClientService`**: integrações externas modeladas como `@Service` sem portas (interfaces) no domínio. O `CoreBancarioClientService` ainda mistura regra (dados padrão de conta) com tratamento técnico de falha.
- **Projeto em módulo único**: sem separação física em camadas, nada impede, em tempo de compilação, que o controller use a entidade JPA ou que o domínio importe Spring.

O **Core Bancário** não existia no estado inicial, então não violava, mas a sua ausência fazia a Plataforma operar com dados falsos (falha 2). A **Mesa de Operações** e o **Mobile** são front-ends sem domínio nem persistência próprios.

Como ficou: o domínio da Plataforma e do Core está em módulos `domain` em Java puro (sem Spring, JPA ou Feign; Lombok apenas em compilação, escopo `provided`). Entidades com fábricas e métodos de negócio, Value Objects imutáveis (`Cpf`, `Saldo`, `AnaliseRisco`...), Domain Service (`MotorRegrasCredito`) e portas (`SolicitacaoCreditoRepository`, `CoreBancarioGateway`...) implementadas por adapters na `infrastructure`, onde ficam as entidades JPA separadas (`SolicitacaoCreditoEntity`) e os mappers.

---

2. Quais componentes possuem rotas de API (endpoints) que violam a semântica do protocolo HTTP e o conceito de Recursos RESTful?

R: O provedor que viola é a **Plataforma de Crédito**, no `CreditoOperacoesNonRestfulController` (`/api/credito`). Todas as 7 rotas são `POST` com **verbos de ação na URI**, um estilo RPC sobre HTTP:

Rota legada	Violação
`POST /solicitarCredito`	Verbo na URI; retorna `200 OK` em vez de `201 Created` com `Location` do recurso criado
`POST /consultarStatusCredito`	Leitura via `POST`, que não é método seguro, idempotente nem cacheável; o filtro (CPF) vai no corpo em vez de query string
`POST /listarTodasSolicitacoes`	Leitura de coleção via `POST` com corpo vazio
`POST /buscarSolicitacaoPorId`	O identificador do recurso vai no corpo, e não na URI (`/solicitacoes/{id}`)
`POST /aprovarSolicitacao` e `POST /reprovarSolicitacao`	A mudança de estado é modelada como "ação" e não como atualização de um recurso; duas rotas para a mesma operação (decisão)
`POST /consultarClienteCore`	Consulta exposta como RPC, com CPF no corpo

Problemas adicionais de semântica HTTP: ausência de versionamento (`/api/v1`), `ID` inexistente respondendo **500** em vez de **404**, nenhum **400** para entradas inválidas e nenhum **409** para conflitos de estado.

Os front-ends **Mesa de Operações** (Feign `PlataformaCreditoFeignClient`) e **Mobile** (`credito.service.ts`) não expõem API, mas **consomem** essas rotas não-RESTful. Por isso a refatoração precisava preservar o contrato antigo.

Como ficou (substantivos no plural, intenção pelo verbo HTTP, status semânticos):

Operação	RESTful
Criar proposta	`POST /api/v1/solicitacoes` → `201` + `Location`
Listar / filtrar por CPF	`GET /api/v1/solicitacoes` · `GET /api/v1/solicitacoes?cpf={cpf}`
Detalhar	`GET /api/v1/solicitacoes/{id}`
Decisão da Mesa	`PUT /api/v1/solicitacoes/{id}/decisao` (sub-recurso)
Cancelar	`DELETE /api/v1/solicitacoes/{id}` → `204`
Conta do cliente no Core	`GET /api/v1/clientes/{cpf}/conta`

O Core Bancário já nasceu RESTful: `/api/v1/clientes` e `/api/v1/contas`, com o sub-recurso `PUT /api/v1/contas/{numeroConta}/saldo`.

---

3. Explique a estratégia técnica adotada para refatorar o componente de forma a implementar o padrão RESTful e o DDD sem quebrar as aplicações cliente legadas.

R: Adotamos uma migração incremental no estilo **Strangler Fig**, com o padrão **Adapter** na borda da aplicação:

1. **Estabilizar o build**: corrigimos o release train do Spring Cloud (2024.0 → 2025.1.x) e os starters do Spring Boot 4, para ter uma base que compila e sobe.
2. **Extrair o domínio puro** (`plataforma-credito-domain`): o agregado `SolicitacaoCredito` passou a ter fábricas (`nova`, `reconstituir`) e comportamentos (`registrarAnaliseAutomatica`, `aprovarPelaMesa`, `reprovarPelaMesa`, `cancelar`) que validam as transições de status. As regras de concessão foram para o Domain Service `MotorRegrasCredito`. As dependências externas viraram **portas** (interfaces) no domínio: `SolicitacaoCreditoRepository`, `CoreBancarioGateway`, `ScrGateway`, `BureauCreditoGateway`, `CrmGateway` e `NotificacaoGateway`.

3. **Infraestrutura como adapters** (`-infrastructure`): a entidade JPA `SolicitacaoCreditoEntity` e o Spring Data ficam isolados, e um mapper converte entre o modelo relacional e o modelo de domínio. O client Feign do Core passou a ser tipado e funciona como **camada anticorrupção**. O fallback com dados fictícios foi removido.

4. **Casos de uso na aplicação** (`-application`): `SolicitacaoCreditoService` orquestra as portas e o motor. Os DTOs são **Java Records** com **Bean Validation**, e as exceções de domínio são traduzidas em exceções de aplicação, porque a apresentação não enxerga o domínio.

5. **Nova API RESTful** (`-presentation`): controllers em `/api/v1` com verbos e status semânticos e um `GlobalExceptionHandler` (`@RestControllerAdvice`) padronizando erros no formato Problem Details (RFC 9457).

6. **Controller Adaptador para o legado**: o `CreditoOperacoesLegadoAdapterController` mantém **as mesmas rotas, verbos, nomes de campos, payloads e status** do contrato histórico (`POST /api/credito/*`, inclusive o `200` do `solicitarCredito`). Ele não tem regra de negócio: só traduz o contrato antigo e **delega para os mesmos casos de uso da API nova**, então legado e RESTful nunca divergem. Toda resposta sinaliza a descontinuação com os cabeçalhos `Deprecation: true` e `Link: <rota-nova>; rel="successor-version"`.

7. **Migração gradual dos clientes**:

    - O **Mobile (Angular)** não teve nenhuma linha alterada e continua funcionando pelo adapter, o que comprova a retrocompatibilidade.
    - A **Mesa (Thymeleaf)** teve só a interface Feign migrada para `GET /api/v1/solicitacoes`, `GET /api/v1/solicitacoes/{id}` e `PUT /api/v1/solicitacoes/{id}/decisao`. Telas e formulários não mudaram.
    - Quando todos os consumidores migrarem, o adapter pode ser removido sem impacto no núcleo.

8. **Governança automatizada das fronteiras**: a estrutura multi-módulo segue `presentation → application → infrastructure → domain`, e o `maven-enforcer-plugin` falha o build se:

    - o `domain` tiver qualquer dependência de compile/runtime;
    - a `infrastructure` depender de `application`/`presentation`;
    - a `presentation` receber `domain`/`infrastructure` no classpath de compilação. Eles são redeclarados com escopo `runtime`, então um `import` direto nem compila, mas o Spring ainda os encontra em execução.

    Um workflow do GitHub Actions executa `mvn verify` a cada push.

Com isso, a alta disponibilidade é preservada: os contratos em produção continuam respondendo enquanto o núcleo é reescrito com DDD e a nova API RESTful passa a coexistir com eles.